

Logic Grid Puzzle Solver – CSP-Based AI System

Team Name : Team42

Team Members :

- Muhammad Zeeshan Abbas - 504504
 - Ben Winter - 50106895
 - Jovaria Khan - 500905
 - Alap Gohar - 502082
 - Amar Semmo - 50140358
 - Aaron Schröder - 50111950
 - Hilal Ebru Duran - 200503037
-

1. Project Overview

This project develops an intelligent **Constraint Satisfaction Problem (CSP) solver** for Logic Grid (Zebra) Puzzles. It converts natural language puzzle descriptions into formal CSP models and solves them efficiently using constraint propagation and heuristic-based search.

2. System Architecture

The system is designed using a modular architecture to ensure clarity, scalability, and maintainability.

Parser Module (parser_test.py) :

The parser processes natural language puzzle descriptions and extracts categories, variables, domains, and constraints. It transforms unstructured text into structured CSP models suitable for automated reasoning.

Solver Module (solver.py) :

This module implements the core CSP solving engine using classical AI techniques:

- AC-3 (Arc Consistency) to prune domains before search
- Backtracking Search to systematically explore solutions

- Minimum Remaining Values (MRV) heuristic to select the most constrained variable first
- Forward Checking to detect conflicts early

Together, these methods significantly reduce the search space and improve efficiency.

Main Execution Module (main.py)

The main module manages the complete pipeline, including loading puzzle datasets, parsing puzzles into CSP format, solving them, and generating competition-ready output files. It supports batch processing of large datasets.

3. Algorithmic Complexity and Optimizations

The solver has a worst-case time complexity of $O(d^n)$, where d is the domain size and n is the number of variables. However, the use of MRV, forward checking, and AC-3 preprocessing greatly reduces practical runtime. Space complexity includes $O(n \times d)$ for domain storage and $O(e)$ for constraint representation.

Key optimizations include early domain pruning, intelligent variable ordering, and immediate conflict detection during search.

4. Key Contributions

- Hybrid constraint propagation using AC-3 and forward checking
 - Heuristic-driven variable selection to minimize backtracking
 - Robust natural language parsing for automatic CSP generation
 - Scalable and modular system design
-

5. Conclusion

This project demonstrates the effectiveness of classical AI techniques in solving complex CSPs with high efficiency and accuracy. The modular architecture ensures extensibility and makes the system applicable beyond logic puzzles to other CSP domains such as scheduling, resource allocation, and configuration problems.